

Actividades Guiadas - Componentes Web 2026

Este documento agrega cuatro actividades guiadas al banco de com sin modificar las guias anteriores. Se penso como complemento pract para aprender haciendo dentro de la misma pagina del proyecto.

Contenido

1. Conversor de temperatura en JavaScript
2. Tarjeta de presentacion Bootstrap
3. Carrito de compras DOM
4. Aprende algo de Bootstrap

Cada actividad incluye:

- objetivo
- explicacion conceptual
- pasos de implementacion
- puntos de validacion

Actividad 1 - Conversor de temperatura en JavaScript

Objetivo

Recibir un valor en grados Celsius, validarlo y convertirlo a Fahrenheit y Kelvin usando JavaScript.

Condiciones de la actividad

- pedir Celsius con prompt o input
- validar que el dato sea numerico
- mostrar Kelvin y Fahrenheit
- repetir o mostrar error si el dato no es valido
- probar con 45 y 14
- subir el resultado al repo logica-programacion-2

Paso a paso

1. Lee el valor escrito por el usuario.
2. Convierte el texto con `Number(valor)`.
3. Valida con `Number.isFinite`.
4. Si falla, muestra un mensaje de error.
5. Si pasa, calcula $\text{Kelvin} = C + 273.15$.
6. Calcula $\text{Fahrenheit} = (C * 9 / 5) + 32$.
7. Imprime ambos resultados en DOM o consola.
8. Verifica $45 \Rightarrow 318.15 \text{ K y } 113 \text{ F}$.
9. Verifica $14 \Rightarrow 287.15 \text{ K y } 57.2 \text{ F}$.

Actividad 1 - HTML copiar y pegar

```
<div id="temperatureTool" class="temperature-tool">
  <div class="unit-buttons">
    <button type="button" class="unit-button is-active"
      data-unit="celsius">C</button>
    <button type="button" class="unit-button"
      data-unit="fahrenheit">F</button>
    <button type="button" class="unit-button"
      data-unit="kelvin">K</button>
  </div>
  <input id="temperatureInput" type="number"
    placeholder="Ingresa un valor">
  <button id="calculateTemperature" type="button">
    Calcular
  </button>
  <p id="temperatureMessage">Unidad seleccionada: celsius</p>
  <div id="temperatureResult"></div>
</div>
<div id="toastContainer"></div>
```

Actividad 1 - CSS copiar y pegar

```
.temperature-tool {  
  display: grid;  
  gap: 12px;  
  padding: 20px;  
  border: 1px solid #cbd5e1;  
  border-radius: 18px;  
}  
.unit-buttons { display: flex; gap: 8px; }  
.unit-button {  
  padding: 10px 14px;  
  border: 1px solid #cbd5e1;  
  border-radius: 999px;  
  background: #ffffff;  
}  
.unit-button.is-active {  
  background: #2563eb;  
  color: #ffffff;  
  border-color: #2563eb;  
}
```

Actividad 1 - JS copiar y pegar

```
class ToastManager {
  constructor(id = 'toastContainer') {
    this.container = document.getElementById(id);
  }
  show(message, time = 2800) {
    const toast = document.createElement('div');
    toast.className = 'toast';
    toast.textContent = message;
    this.container?.appendChild(toast);
    setTimeout(() => toast.remove(), time);
  }
}

document.addEventListener('DOMContentLoaded', () => {
  const buttons = [...document.querySelectorAll('.unit-button')];
  const input = document.getElementById('temperatureInput');
  const result = document.getElementById('temperatureResult');
  const message = document.getElementById('temperatureMessage');
  const calc = document.getElementById('calculateTemperature');
  const toast = new ToastManager();
  let selectedUnit = 'celsius';
  buttons.forEach((button) => button.addEventListener('click', () => {
    selectedUnit = button.dataset.unit;
    buttons.forEach((item) => item.classList.remove('is-active'));
    button.classList.add('is-active');
    message.textContent = 'Unidad seleccionada: ${selectedUnit}';
  }));
  calc?.addEventListener('click', () => {
    const value = Number(input.value);
    if (!Number.isFinite(value)) return;
    let celsius = value;
    if (selectedUnit === 'fahrenheit') celsius = (value - 32) * 5 / 9;
    if (selectedUnit === 'kelvin') celsius = value - 273.15;
    const fahrenheit = (celsius * 9 / 5) + 32;
    const kelvin = celsius + 273.15;
    result.innerHTML = `C: ${celsius.toFixed(2)}<br>F: `
```

Actividad 2 - Tarjeta de presentacion Bootstrap

Objetivo

Construir una card personal inspirada en Bootstrap y adaptarla al estilo visual de este proyecto.

Que se practica

- estructura tipo card
- jerarquia visual con avatar, nombre, rol y enlaces
- badges para tecnologias
- composicion responsiva
- interaccion extra con flip frontal y trasero

Paso a paso

1. Crea un contenedor principal para la actividad.
2. Agrega un bloque interno que gire con transform 3D.
3. Diseña la cara frontal con avatar, nombre y descripción.
4. Diseña la cara trasera con tecnologías y herramientas.
5. Agrega botones para ver tecnologías y volver.
6. En CSS usa perspective, preserve-3d y backface-visibility.
7. En JS agrega o quita la clase is-flipped.
8. Prueba contraste, espaciado y responsive en ambos temas.

Actividad 2 - HTML copiar y pegar

```
<article class="profile-lab">
  <div class="profile-lab__flip" id="profileLabFlip">
    <section class="profile-lab__face profile-lab__face--front">
      <div class="profile-lab__avatar">CM</div>
      <h4>Carlos Madero</h4>
      <p class="profile-lab__role">Full Stack Java Developer</p>
      <button id="showProfileTech" type="button">
        Ver tecnologías
      </button>
    </section>
    <section class="profile-lab__face profile-lab__face--back">
      <h4>Tecnologías</h4>
      <button id="hideProfileTech" type="button">
        Volver
      </button>
    </section>
  </div>
</article>
```

Actividad 2 - CSS copiar y pegar

```
.profile-lab { max-width: 340px; perspective: 1200px; }
.profile-lab__flip {
  position: relative;
  min-height: 320px;
  transform-style: preserve-3d;
  transition: transform 0.8s ease;
}
.profile-lab__flip.is-flipped { transform: rotateY(180deg); }
.profile-lab__face {
  position: absolute;
  inset: 0;
  display: grid;
  gap: 12px;
  padding: 24px;
  border: 1px solid #cbd5e1;
  border-radius: 20px;
  background: #ffffff;
  backface-visibility: hidden;
}
.profile-lab__face--back { transform: rotateY(180deg); }
```


Actividad 2 - JS copiar y pegar

```
document.addEventListener('DOMContentLoaded', () => {  
  const flip = document.getElementById('profileLabFlip');  
  const show = document.getElementById('showProfileTech');  
  const hide = document.getElementById('hideProfileTech');  
  show?.addEventListener('click', () => {  
    flip?.classList.add('is-flipped');  
  });  
  hide?.addEventListener('click', () => {  
    flip?.classList.remove('is-flipped');  
  });  
});
```

Actividad 3 - Carrito de compras DOM

Objetivo

Practicar manipulacion del DOM creando una mini tienda con carrito contador, total acumulado y borrado de items.

Que se practica

- querySelectorAll y addEventListener
- dataset para leer nombre y precio
- createElement y appendChild
- actualizacion del badge
- calculo del total
- eliminacion individual y vaciado completo

Paso a paso

1. Crea varios productos con boton Agregar.
2. Guarda nombre y precio en data-name y data-price.
3. Crea la lista del carrito y el badge del contador.
4. Al agregar, crea un li dinamico con boton Eliminar.
5. Suma el precio al total y actualiza la interfaz.
6. Cuando se elimina un item, resta el valor y borra la fila.
7. El boton Vaciar debe reiniciar lista, badge y total.
8. Verifica que todo siga correcto al mezclar varias acciones.

Actividad 3 - HTML copiar y pegar

```
<div class="shop-lab">
  <div class="shop-lab__topbar">
    <strong>Mini tienda</strong>
    <span class="shop-lab__badge" id="shopBadge">0</span>
  </div>
  <div class="shop-lab__grid">
    <article class="shop-product">
      <h4>Pack IPA</h4>
      <p>$18,000</p>
      <button class="shop-add-button" type="button"
        data-name="Pack IPA" data-price="18000">Agregar</button>
    </article>
    <article class="shop-product">
      <h4>Cheesecake</h4>
      <p>$11,000</p>
      <button class="shop-add-button" type="button"
        data-name="Cheesecake" data-price="11000">Agregar</button>
    </article>
  </div>
  <div class="shop-cart">
    <div class="shop-cart__head">
      <h4>Carrito</h4>
      <button id="clearShopCart" type="button">Vaciar</button>
    </div>
    <p id="shopEmptyMessage">Tu carrito esta vacio.</p>
    <ul id="shopCartList"></ul>
    <strong id="shopCartTotal">$0</strong>
  </div>
</div>
<div id="toastContainer"></div>
```

Actividad 3 - CSS copiar y pegar

```
.shop-lab { display: grid; gap: 18px; }
.shop-lab__topbar {
  display: flex;
  justify-content: space-between;
  align-items: center;
}
.shop-lab__badge {
  min-width: 34px;
  padding: 6px 10px;
  border-radius: 999px;
  background: #2563eb;
  color: #ffffff;
  text-align: center;
}
.shop-lab__grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(180px, 1fr));
  gap: 14px;
}
.shop-product, .shop-cart {
  padding: 16px;
  border: 1px solid #cbd5e1;
  border-radius: 16px;
}
```

Actividad 3 - JS copiar y pegar

```
class ShoppingCartActivity {
  constructor() {
    this.buttons = [...document.querySelectorAll('.shop-add-button')];
    this.list = document.getElementById('shopCartList');
    this.badge = document.getElementById('shopBadge');
    this.total = document.getElementById('shopCartTotal');
    this.empty = document.getElementById('shopEmptyMessage');
    this.clearButton = document.getElementById('clearShopCart');
    this.itemsCount = 0;
    this.totalAmount = 0;
    this.init();
  }
  init() {
    this.buttons.forEach((button) => {
      button.addEventListener('click', () => {
        this.addItem(button.dataset.name, Number(button.dataset.price));
      });
    });
    this.clearButton?.addEventListener('click', () => this.clear());
    this.renderSummary();
  }
  addItem(name, price) {
    const item = document.createElement('li');
    item.className = 'shop-cart__item';
    item.innerHTML = `<div><strong>${name}</strong><p>${price}</p></div><button type="button">X</button>`;
    item.querySelector('button')?.addEventListener('click', () => {
      item.remove();
      this.itemsCount -= 1;
      this.totalAmount -= price;
      this.renderSummary();
    });
    this.list?.appendChild(item);
    this.itemsCount += 1;
    this.totalAmount += price;
  }
}
```

Actividad 4 - Aprende algo de Bootstrap

Objetivo

Entender que parte resuelve Bootstrap con clases listas y que parte igual debes dominar con HTML, CSS y JavaScript propios.

Que se practica

- lectura de layout con container, row y col
- analisis de card, badge y boton
- traduccion de patrones Bootstrap a CSS nativo
- comparacion entre clases de utilidad y decisiones reales
- reconstruccion de una seccion tipo pricing

Paso a paso

1. Observa la estructura general del bloque y separa layout de estilo
2. Identifica donde Bootstrap aporta grid, padding y jerarquia visual.
3. Reescribe el patron con clases propias y variables CSS.
4. Crea tres cards de planes con un plan destacado.
5. Agrega un badge, un precio fuerte y una lista de beneficios.
6. Usa grid responsive para mover las cards sin romper la lectura.
7. Cambia colores, radios y sombras para probar variantes.
8. Compara el resultado con la solucion guiada de la pagina.

Actividad 4 - HTML copiar y pegar

```
<section class="pricing-lab">
  <article class="pricing-lab__card">
    <span class="pricing-lab__badge">Basico</span>
    <h4>Starter</h4>
    <p class="pricing-lab__price">$19<span>/mes</span></p>
    <ul class="pricing-lab__list">
      <li>1 proyecto</li>
      <li>Soporte por correo</li>
      <li>Componentes base</li>
    </ul>
    <button type="button">Elegir plan</button>
  </article>
  <article class="pricing-lab__card pricing-lab__card--featured">
    <span class="pricing-lab__badge">Popular</span>
    <h4>Pro</h4>
    <p class="pricing-lab__price">$49<span>/mes</span></p>
    <ul class="pricing-lab__list">
      <li>10 proyectos</li>
      <li>Soporte prioritario</li>
      <li>Layouts premium</li>
    </ul>
    <button type="button">Elegir plan</button>
  </article>
</section>
```

Actividad 4 - CSS copiar y pegar

```
.pricing-lab {  
  display: grid;  
  grid-template-columns: repeat(auto-fit, minmax(220px, 1fr));  
  gap: 16px;  
}  
.pricing-lab__card {  
  padding: 20px;  
  border: 1px solid #cbd5e1;  
  border-radius: 20px;  
  background: #ffffff;  
  box-shadow: 0 18px 40px rgba(15, 23, 42, 0.12);  
}  
.pricing-lab__card--featured {  
  border-color: #4f46e5;  
  transform: translateY(-6px);  
}  
.pricing-lab__badge {  
  display: inline-flex;  
  padding: 6px 10px;  
  border-radius: 999px;  
  background: #e0e7ff;  
  color: #3730a3;  
}  
.pricing-lab__price { font-size: 2rem; font-weight: 800; }
```


Actividad 4 - JS copiar y pegar

```
document.addEventListener('DOMContentLoaded', () => {  
  const cards = [...document.querySelectorAll('.pricing-lab__card')];  
  cards.forEach((card) => {  
    const button = card.querySelector('button');  
    button?.addEventListener('click', () => {  
      cards.forEach((item) => item.classList.remove('pricing-lab__card--featured'));  
      card.classList.add('pricing-lab__card--featured');  
    });  
  });  
});
```